

Receipt Rail

A chat-first receipt capture, auto-reconciliation and statement-submission layer bolted onto Vippy Spend Tracker.

IMPLEMENTATION PLAN · v1.0 · 18 Aug 2026

~45min

MANUAL EFFORT TODAY, PER CYCLE

~3min

AFTER RECEIPT RAIL

\$2.30/mo

ADDED RUNNING COST

THE PROBLEM

Physical invoices for the VIP Industries corporate card get lost between the purchase and the accounts department. Every statement cycle ends in a manual hunt: match paper to line items, scan, name files, email one by one.

THE FIX

Photograph the bill the moment you get it and send it to a Telegram bot. It reads the bill, matches it to the exact card transaction already parsed from your HDFC email alerts, files it against the right statement cycle, and — on your approval — emails accounts one clean, indexed PDF bundle.

WHAT THIS DOCUMENT COVERS

01–03	Concept & system map	Before/after, full architecture, where it plugs into today's app
04–10	The flows	Chat journey, ingest, extraction & model cost, foreign currency, matching, trips & backfill
11–14	The cycle	Statement cycle, reconciliation & tie-out, the CSV package, delivery
15–18	The build	Data model, API surface, tool stack & cost, infrastructure options, security
19–23	Decisions	Telegram vs iMessage, phased roadmap, risks, what I need from you

From paper chase to a two-second photo

Nothing about how you spend changes. The only new habit is: **snap the bill, send it to the bot.** Everything downstream is automatic.

TODAY

Manual, lossy, month-end panic

- 1 Swipe card. HDFC alert lands in Gmail. App parses it. **Paper bill goes in a pocket.**
- 2 Weeks pass. Bill is lost, faded, or in another jacket.
- 3 Statement arrives. You reconstruct 40+ line items from memory.
- 4 Scan whatever survived, rename files, write a covering mail.
- 5 Accounts replies asking which receipt belongs to which charge.

Failure rate: receipts recovered only for the charges you happened to keep paper for.

WITH RECEIPT RAIL

Capture at the point of spend

- 1 Swipe card. HDFC alert lands in Gmail → app parses it (this already works).
- 2 **Photo the bill → send to Telegram bot.** Takes 2 seconds at the counter.
- 3 Bot reads amount/date/merchant/GSTIN, matches it to the transaction, confirms in chat.
- 4 Daily nudge lists charges still missing a receipt so nothing rots until month-end.
- 5 Cycle closes → bot builds the bundle → you tap **Send to accounts** → done.

Failure mode is now visible: anything unmatched is flagged and chased, not silently missing.

THE THREE JOBS THIS LAYER DOES

JOB 1

Capture

Move the receipt off paper into durable storage the instant it exists, from whatever phone you're holding.

JOB 2

Reconcile

Bind each receipt to the exact card transaction — the hard part accounts actually cares about.

JOB 3

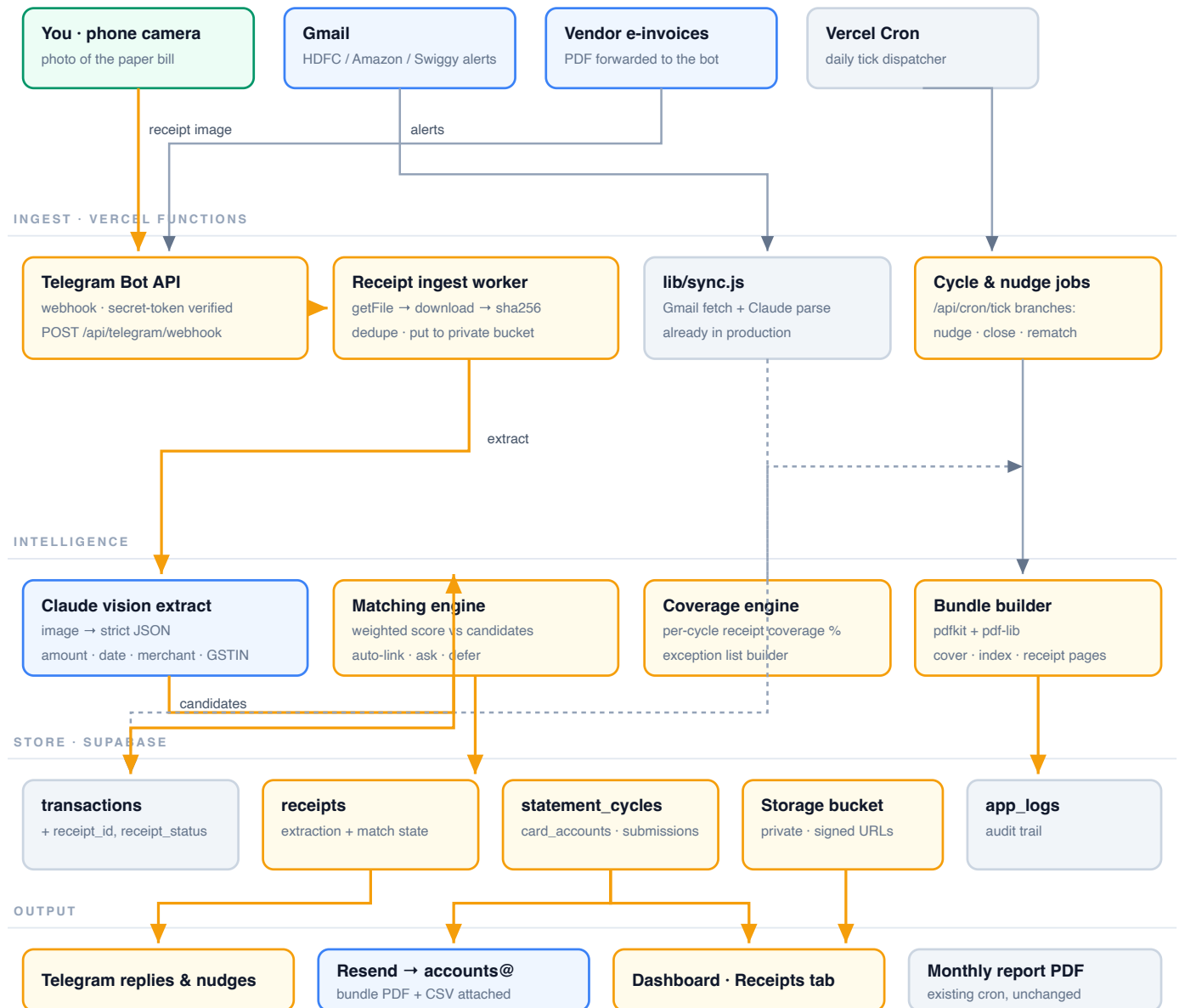
Submit

Package a statement cycle into one indexed, auditable PDF and deliver it with an approval gate.

Every moving part, end to end

■ Exists today ■ New in Receipt Rail ■ External service ■ Human action

SOURCES



One new front door. Telegram webhook is the only public surface added; it is authenticated by a secret header and a chat-ID allowlist.

Reuses the existing brain. Transactions still come from Gmail + Claude exactly as they do now. Receipt Rail never re-parses bank mail.

Two-way street. The bot both receives receipts and pushes reminders back, which is what actually kills the month-end scramble.

Bolt-on, not rewrite

Receipt Rail is additive. Existing sync, dashboard, reports and the monthly-report cron keep working untouched. Below is the honest diff against the repo as it stands.

EXISTING FILE	VERDICT	WHAT HAPPENS
lib/sync.js	1 hook added	After a successful insert batch, fire rematchPendingReceipts(userId) so receipts that arrived before the bank email get bound the moment the transaction appears.
lib/gmail.js	untouched	No change. Same queries, same parsing.
lib/report-pdf.js	pattern reused	The pdfkit + QuickChart approach is copied into a new lib/bundle-pdf.js. The monthly report itself is unchanged.
lib/monthly-report.js	pattern reused	Resend delivery pattern copied into lib/submission-mail.js, with an approval gate in front of it.
lib/supabase.js	must extend	Today it only builds an anon-key client. Storage writes and cross-user cron work need a second getSupabaseAdmin() using SUPABASE_SERVICE_ROLE_KEY, server-only.
lib/logger.js	untouched	Reused as-is with new sources: telegram, receipt, match, submission.
transactions table	3 columns	receipt_id, receipt_status, declared_reason. The legacy has_receipt boolean becomes a generated view of receipt_status.
app/page.js + tabs	1 new tab	Receipts tab returns — but as a reconciliation console, not the old checkbox UI that was removed in 3e26a4f.
vercel.json crons	consolidate	Two cron slots are already used. See the constraint below.

CONSTRAINT · CRON SLOTS

Vercel's Hobby plan caps you at **2 cron jobs, once-daily**. /api/cron/sync and /api/cron/monthly-report already occupy both. Receipt Rail needs three more schedules.

Fix without upgrading: replace both entries with one /api/cron/tick dispatcher that runs daily and branches on the date — sync and nudge always, statement reconciliation on the 18th, submission build on the 23rd. One slot, five jobs, and every job stays independently testable via ?job=.

CONSTRAINT · SUPABASE KEY

The app currently ships only SUPABASE_ANON_KEY. A private receipts bucket cannot be written with the anon key under sane RLS, and the cron already reads **all** users' rows — which today works only because RLS is effectively open.

Do this in Phase 0: add SUPABASE_SERVICE_ROLE_KEY (server env only, never NEXT_PUBLIC_), turn RLS on for receipts/submissions, and route all storage + cron writes through the admin client.

NEW FILES

```
app/api/telegram/webhook/route.js
app/api/receipts/route.js
app/api/receipts/[id]/match/route.js
app/api/cycles/route.js
app/api/submissions/route.js
app/api/submissions/[id]/send/route.js
app/api/cron/tick/route.js
```

```
lib/telegram.js // send, edit, getFile,
keyboards
lib/receipt-vision.js // Claude image → JSON
lib/matcher.js // scoring + decisions
lib/cycles.js // statement windows
lib/bundle-pdf.js // submission bundle
lib/submission-mail.js
app/components/receipts/ReceiptsTab.jsx
```

The whole product, in four conversations

A · ONE-TIME LINKING

/start VIP-7K42

Linked. 🙌 Khush — VIP Industries corporate card ••4821.

Send me a photo of any bill and I'll file it against the right charge. Type /help anytime.

Cycle open: **4 Aug → 3 Sep**
18 charges · ₹1,42,380 · 11 receipts on file (61%)

Code is generated in Settings → Receipt Bot. It binds your Telegram chat ID to your user_id. Nobody else's chat can talk to the bot.

B · THE HAPPY PATH

IMG_4821.jpg

Reading...

✅ **Matched · 96**
Indian Oil · ₹3,400.00 · 12 Aug, 18:42
→ Txn #2841 HDFC ••4821 · IOCL PETROL
GSTIN 27AAAC1681G1ZN · Inv 5512

[↩ Wrong match](#) [👉 Change category](#)

Score ≥ 75 with a clear gap to the runner-up → auto-linked, undo always available.

C · AMBIGUITY — THE BOT ASKS INSTEAD OF GUESSING

restaurant bill · ₹2,480

🤔 **Two charges fit** — ₹2,480 on 09 Aug.

1 · 09 Aug 21:14 · SOCIAL OFFLINE · ₹2,480 (88)

2 · 09 Aug 21:19 · SWIGGY · ₹2,480 (74)

* None of these — hold it

1

Filed against Txn #2803. 👍

D · THE NUDGE THAT ENDS MONTH-END PANIC

📌 **Daily check · 6 charges without a receipt**
14 Aug · Taj Santacruz · ₹8,940
13 Aug · Indigo · ₹12,300
11 Aug · Croma · ₹4,199
+3 more under ₹500 (auto-waived)

[📎 Send now](#) [🚫 No bill exists](#) [🕒 Remind tomorrow](#)

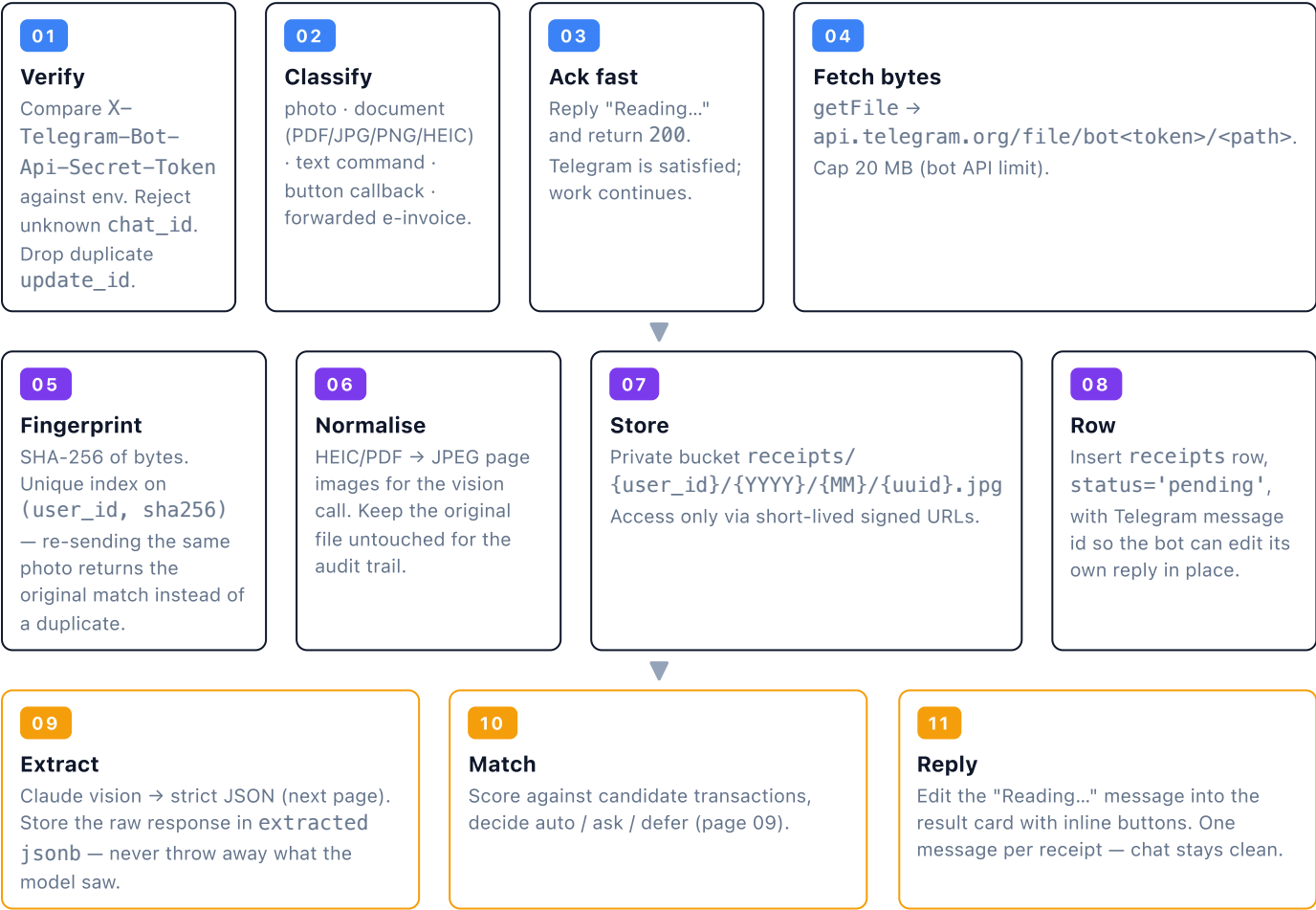
🕒 **Cycle closes in 3 days.** Coverage 78%.
Tap /submit when ready — I'll build the bundle for your approval first.

COMMAND SURFACE

/start <code>	Link this chat to your dashboard account	/status	Current cycle coverage + totals
/missing	List charges still lacking a receipt	/unmatched	Receipts waiting for a transaction
/submit	Build the bundle and preview it here	/last	Show + undo the last match
/note <text>	Attach a business-purpose note to the last receipt	/help	Command list

Ingest, step by step

Telegram's webhook must be answered in under a few seconds or it retries. So the webhook does the cheap work and acknowledges immediately; the expensive work runs after the response is queued.



FAILURE HANDLING — EVERY BRANCH HAS A DEFINED END STATE

FAILURE	BEHAVIOUR
Vision call errors / rate-limited	Exponential backoff, 3 tries; then row stays pending and the daily job retries. Chat says "couldn't read it — I'll retry".
Image unreadable / not a bill	doc_type='other' → bot asks "Is this a receipt? Enter amount + date manually" with a modal-style prompt.
Zero candidate transactions	status='unmatched' → parked in the deferred pool, retried after every Gmail sync for 30 days.
File > 20 MB	Rejected with instructions to send as a photo, not an uncompressed document.
Webhook retried by Telegram	update_id dedupe table makes the whole handler idempotent.

WHY NOT JUST EMAIL RECEIPTS TO THE APP?

You could forward bills to a Gmail alias and let the existing sync pick them up. It fails on the thing that matters: **no feedback loop**. Email can't tell you "this didn't match, pick one of these two" in the two seconds you're still standing at the counter, and it can't nudge you at 8pm about the three charges still bare.

Chat is the right medium because reconciliation is a **conversation**, not a drop-box.

Reading the bill

Same provider you already pay for (@anthropic-ai/sdk is in package.json), same discipline as lib/sync.js: a strict system prompt, JSON-only output, and a deterministic safety net over the model's answer.

OUTPUT CONTRACT

```
// lib/receipt-vision.js → extractReceipt(buf)
{
  "doc_type": "tax_invoice", // invoice|pos_slip|e_invoice|other
  "merchant": "Indian Oil Corporation",
  "merchant_raw": "IOCL RETAIL OUTLET, ANDHERI E",
  "total": 3400.00,
  "subtotal": 2881.36,
  "tax_total": 518.64,
  "currency": "INR",
  "date": "2026-08-12",
  "time": "18:42",
  "invoice_no": "5512",
  "gst_in": "27AAACI1681G1ZN",
  "payment_mode": "card",
  "card_last4": "4821",
  "line_items": [{ "desc": "XP95 Petrol 34.2L", "amount": 3400 }],
  "confidence": 0.94,
  "unreadable_fields": []
}
```

PROMPT RULES THAT EARN THEIR KEEP

- Total ≠ subtotal.** Always return the **amount actually charged to the card** — the grand total after tax, tip and rounding. That is the only number the bank alert will agree with.
- Never invent.** Anything illegible goes in unreadable_fields and stays null. A null the bot asks about is cheap; a hallucinated GSTIN in a filing is not.
- Indian formats.** DD/MM/YYYY, lakh/crore digit grouping, "Rs."/"₹"/"INR", GSTIN regex validation, CGST+SGST split.
- Card tail is gold.** If the slip prints the last 4 digits, extraction confidence and match score both jump — it proves the bill belongs to the **company** card, not your personal one.

DETERMINISTIC SAFETY NET AFTER THE MODEL RETURNS

- Arithmetic check**
subtotal + tax ≈ total (± ₹1) else drop confidence 0.2
- GSTIN checksum**
15-char format + state code validation; invalid → null it
- Date sanity**
Future dates or >120 days old → flag for confirmation
- Currency guard**
Non-INR → convert flag raised, never silently matched

Where the model choice lands: accuracy is the priority, so every receipt is read **twice by two different model families** and conflicts are adjudicated by a third. Full strategy and costs on the next page.

You can't buy 100% from a model. You can engineer it.

The honest position. No vision model returns 100% on creased thermal slips, faded ink and handwritten tips — not Claude, not GPT, not a dedicated OCR service. Anyone quoting 100% is quoting a benchmark, not your shoebox. **But 100% accuracy at the point of submission is achievable**, because every rupee in the package is checked against HDFC's own statement and anything that doesn't reconcile blocks the send. Accuracy is a property of the **pipeline**, not the model.

FOUR LAYERS, EACH CATCHING WHAT THE ONE BEFORE IT MISSED

L1

Two models, independently
Every receipt read by **Sonnet 5** and **GPT-5.6 Terra** in parallel. Agreement on total + date + merchant is strong evidence; disagreement is a flag, not a coin toss.

L2

Tiebreak on conflict
They disagree → **Opus 5** adjudicates. Fires on ~15% of receipts, so the flagship price is paid only where it earns its keep.

L3

Deterministic checks
Arithmetic tie-out, GSTIN/VAT checksum, currency sanity, date plausibility. Code, not a model — it never has an off day.

L4

The statement
The real guarantee. Every amount must equal a line on HDFC's statement. Misread ₹3,400 as ₹8,400 and no statement line matches — the cycle blocks, the discrepancy is named in chat, you fix it in one tap.

The bank is the ground truth, and nothing reaches accounts without agreeing with it to the rupee. That is a stronger promise than any model benchmark — and it is the layer that actually delivers what you asked for.

WHAT EACH OPTION COSTS — ACCURACY-FIRST, CHEAP-SECOND

MODEL	\$/MTOK IN·OUT	PER RECEIPT	ROLE IN THIS DESIGN
Claude Sonnet 5 PRIMARY A	\$2 · \$10	\$0.0078	Strongest structured extraction at this tier; already integrated.
GPT-5.6 Terra PRIMARY B	\$2 · \$12	\$0.0084	Independent second read on your existing OpenAI key. A different model family is the point — correlated errors defeat consensus.
Claude Opus 5 TIEBREAK	\$5 · \$25	\$0.019	Adjudicates disagreements only, ~15% of receipts.
Sarvam Vision SPECIALIST	₹0.50 / page	\$0.0057	Third opinion on handwritten or Indic-script Indian slips. No European languages.
Luna · Nano · Flash-Lite	\$0.20–0.25 in	\$0.0009	Cheapest on the board, and OpenAI's small tier genuinely undercuts Anthropic's — but a single cheap read is exactly the trade you told me not to make. Kept only as failover if a provider is down.
Mistral OCR 4 / Document AI	\$4–30 / 1k pages	\$0.004–0.03	OCR only — still needs a model on top, adding cost and a failure point without adding accuracy.

THE BILL FOR INSISTING ON ACCURACY

Sonnet 5 · every receipt	\$0.0078
GPT-5.6 Terra · every receipt	\$0.0084
Opus 5 · ~15% tiebreak	\$0.0029
Per receipt	\$0.019
120 receipts / month	≈ \$2.30

₹200 a month to read everything twice and adjudicate every conflict — about ₹190 more than a single cheap read. One wrong figure reaching accounts costs more than a year of that difference, in credibility rather than time.

Verify on day one: run 20 real bills through both models, record the agreement rate, then tune routing on that data rather than this table.

TWO DECISIONS WORTH STATING PLAINLY

You already hold both keys. OpenAI is genuinely cheaper at the small-model tier — Luna's \$0.20/\$1.20 undercuts everything. At the **flagship** tier it reverses: Terra's \$2/\$12 is marginally dearer than Sonnet 5's \$2/\$10. Using both anyway, price stops deciding and consensus becomes the point.

Leave lib/sync.js alone. Its prompt is tuned in production for refund detection, merchant canonicalisation and dedupe. Swapping models to consolidate providers buys nothing and risks a missed refund landing in a submission — and split providers mean one outage cannot stop the pipeline.

Your Europe trip breaks naive matching — here's the fix

The core problem. A €48.50 dinner in Lisbon does not appear on your HDFC alert as €48.50, or as any round conversion of it. It appears as ₹4,7xx — Visa's wholesale rate, plus HDFC's ~3.5% foreign-currency markup, plus 18% GST on that markup, posted **1–3 days later** than the date printed on the bill. Exact-amount matching, which is 40 of the 100 match points, fails on **every single one** of your ~20 European receipts.

HOW A FOREIGN CHARGE IS RECONSTRUCTED

1

Extract in origin currency
Model returns total: 48.50, currency: "EUR", country: "PT" — never converted by the model.

2

Fetch the day's rate
ECB daily reference rate for the receipt date via Frankfurter — free, no key. Cached in fx_rates, fetched once per currency-day.

3

Model the markup band
expected₹ = €48.50 × rate × [1.000 ... 1.060]
network spread ~0.5–1% + markup 3.5% + 18% GST on markup.

4

Score against the band
Inside band → full amount points. Within 1.5% outside → partial. The learned markup for your card tightens the band over time.

Worked example. €48.50 · 12 Jun · ECB EURINR 94.10 → base ₹4,563. Band ₹4,563–₹4,837. HDFC posts ₹4,751 on 14 Jun → inside band at +4.1%, which is exactly the expected markup. Amount points awarded, and ₹4,751 — the amount actually charged — is what goes to accounts. The euro figure is shown alongside as evidence only.

RESCORING FOR FOREIGN RECEIPTS

SIGNAL	INR	FOREIGN
Amount	40	30
Date (window)	25 ±7d	20 -1...+5d
Merchant	20	25
Country / city	—	10
Card last 4	10	10
Time	5	5

Amount loses weight because it is now an estimate; merchant name gains it, because a Lisbon restaurant descriptor on the HDFC alert is highly distinctive. Auto-match threshold is raised to 80 for foreign receipts.

Settlement lag is real. Foreign charges post 1–3 business days after the swipe, and a Saturday dinner in Prague can post the following Tuesday. Hence the asymmetric -1...+5 day window rather than ±7.

EVERYTHING ELSE THAT DIFFERS OUTSIDE INDIA

Decimal formats
1.234,56 (DE/IT/ES/PT/NL) vs 1,234.56 (UK/IE) vs 1 234,56 (FR/CZ). Single biggest silent-error source. Explicit prompt rule + a deterministic reparse keyed to the detected country.

Date order
DD/MM across the EU, MM/DD if a US vendor sneaks in. Unambiguous when day > 12; otherwise resolved by country, and flagged for confirmation when it still can't be.

Tax identity
gst_in generalises to tax_id + tax_id_type (GSTIN · EU_VAT · UK_VAT · other), with per-country VAT-number format validation.

Tax labels & languages
IVA · TVA · MwSt · BTW · MOMS · DPH · VAT. Receipts in PT/ES/FR/DE/IT/NL/CS. merchant_raw is kept verbatim; merchant is the normalised English form.

DCC — THE EASY CASE
If the terminal offered "pay in INR?" and you accepted, the slip prints **both** currencies. The model is told to capture the second amount as dcc_amount_inr — and then an **exact** match is available again, no FX modelling needed. (It also means you paid a worse rate; the bot quietly notes it.)

MARKUP & GST FEE ROWS
HDFC often posts the markup fee and its GST as **separate** line items. These are auto-classified as fee, linked to the parent charge, and shown indented under it in the bundle — so accounts sees ₹4,751 + ₹166 markup + ₹30 GST as one story, not three orphan rows.

RECLAIMABLE EU VAT
A Europe trip at 19–23% VAT on ~20 bills is real money. Once tax_breakdown is extracted, the bundle can carry a **reclaimable VAT summary** by country — the input your finance team needs for a 13th-Directive refund claim. Nearly free to generate; potentially the highest-value page in the document.

Binding a receipt to the right charge

This is the part that makes or breaks the product. A wrong auto-match is worse than no match, so the engine is **deliberately conservative**: it only decides alone when the evidence is overwhelming, and it always leaves an undo.

CANDIDATE WINDOW

Pull transactions for this `user_id` where `is_refund = false`, date within **±7 days** of the receipt date, and amount within **±5%** or **±₹20** of the receipt total. Typically 0–4 rows.

WEIGHTED SCORE (MAX 100)

SIGNAL	PTS	RULE
Amount	40	exact 40 · ±0.5% 32 · ±2% or ±₹5 22 · else 0
Date	25	same day 25 · ±1d 19 · ±2–3d 11 · ±4–7d 4
Merchant	20	normalised token overlap + trigram similarity on <code>merchant_raw</code> vs the HDFC descriptor
Card last 4	10	exact match on the printed tail
Time	5	within 30 min of <code>txn_time</code>
Penalty	–30	transaction already has a receipt attached

Amount is dominant on purpose. In practice the bank alert amount is authoritative and identical to the bill total; date and merchant exist to break ties, not to carry the decision.

DECISION TREE

AUTO

top ≥ 75 and (top – second) ≥ 15

Link it. Post the result card with an **Wrong match** undo button. `matched_by='auto'`.



ASK

top ≥ 45, or a close runner-up

Show the top 3 as inline buttons with their scores plus "None of these". Your tap writes `matched_by='user'` — and is logged as training signal.



DEFER

no candidate ≥ 45

Park as **unmatched**. Re-run automatically after every Gmail sync for 30 days — this is the common case when you photograph the bill **before** HDFC sends the alert.



DUPLICATE

sha256 already seen

Reply with the existing match instead of creating a second row. Zero cost, no vision call.

THE FOUR HARD CASES, AND HOW EACH IS HANDLED

Receipt before the alert	Deferred queue + post-sync rematch hook in <code>lib/sync.js</code> . Solved without you doing anything.	One bill, split charges	Two cards / part-cash. Bot offers Link to multiple — <code>receipt_transactions</code> is a join table, not a foreign key, precisely for this.
Tip added after the slip	Bill ₹2,000, card charged ₹2,200. Amount band ±5% covers most tips; beyond that the bot asks and remembers the merchant as a "tip-prone" one.	Two identical amounts, same day	Never auto-matched (gap rule blocks it). You pick from buttons; the merchant name on the slip usually makes it obvious.

Every decision is auditable. `receipts.match_score`, `matched_by` (auto|user|admin|rematch) and an `app_logs` entry are written for every link and every undo. If accounts ever queries a bundle, you can show exactly why receipt R-014 sits against charge #2841.

The 20 European bills already sitting on your desk

Steady-state capture is one photo at a time. A past trip is the opposite problem: a stack of bills, all at once, against transactions that landed in the database weeks ago. Feeding those through the chat flow one by one would take an hour and defeat the point.

BULK IMPORT PATH

- 1 Declare the trip.** /trip Europe · 8–19 Jun 2026 in chat, or create it in the dashboard. Sets the date window and marks the receipts as travel.
- 2 Dump everything.** Telegram media groups take 10 images per send; the dashboard takes an unlimited multi-file drop. Both land in the same queue.
- 3 Batch extraction.** None of this is latency-sensitive, so it goes through the **Batch API at 50% off** — 20 receipts for about **\$0.04** — instead of 20 live calls.
- 4 Match against the trip window,** with the foreign-currency scoring from page 08 and the trip's own transactions as the candidate pool. Far fewer false candidates than a blind ±7-day search.
- 5 One review screen, not 20 chats.** A single table: proposed pairing, score, confidence, and every ambiguous one flagged. Confirm the lot in one pass, fix the two that are wrong.

Why a review screen and not chat here. Chat is right when the decision is one tap in the moment. Twenty simultaneous decisions want a table you can scan — this is precisely where the dashboard earns its place alongside the bot.

WHAT A TRIP ADDS TO THE SUBMISSION

BUNDLE · TRIP SECTION		
EUROPE · 8–19 JUN 2026		
Portugal · Spain · France · Czechia		
Portugal · 6 receipts	€412.80	₹40,4xx
Spain · 5 receipts	€388.10	₹38,0xx
France · 6 receipts	€511.25	₹50,1xx
Czechia · 3 receipts	CZK 6,240	₹24,5xx
Forex markup + GST		₹6,1xx
Reclaimable VAT ≈ €212 across 4 jurisdictions		

Each receipt page in the bundle is stamped with both figures — €48.50 · ₹4,751 @ 97.96 incl. markup — so an accountant can tie the euro bill to the rupee charge without doing the arithmetic themselves. That single line is what stops the "which receipt is this?" reply email.

Retro-matching caveat, stated plainly. For a trip already in the past, some bills will have no findable counterpart — a cash payment, a charge on a different card, a bill the restaurant printed twice. Expect ~80–90% to bind cleanly and to declare the rest by hand. That is still twenty minutes instead of an afternoon.

TRIP DETECTION, SO YOU DON'T HAVE TO REMEMBER

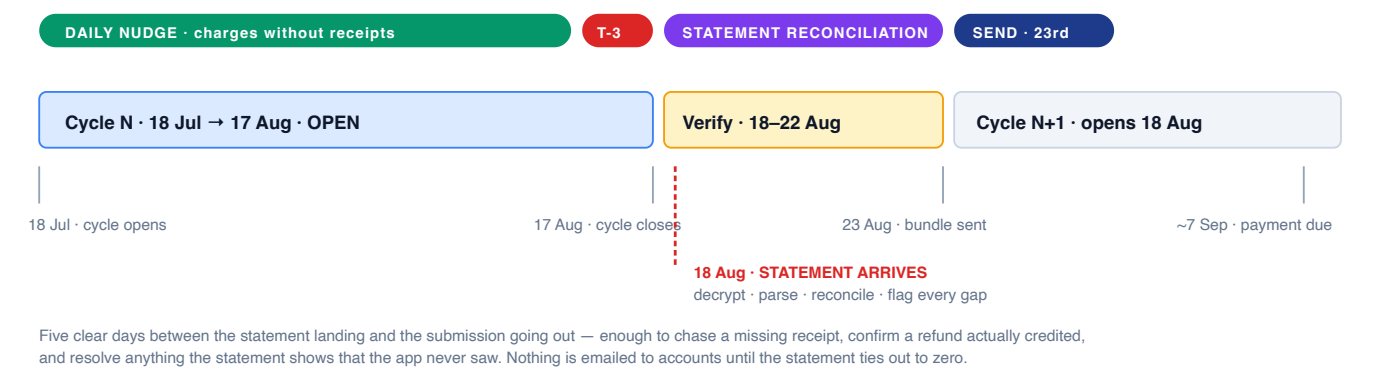
Auto-proposed. Three or more foreign-currency charges inside a 14-day window → the bot offers "Looks like a trip, 8–19 Jun, 4 countries. Name it?" You accept or ignore.

Reminded on return. Trip closes → one nudge listing every foreign charge still bare, with country flags, sent while the stack of paper is still in your bag rather than three weeks later.

Submitted separately if you want. A trip can be bundled on its own for a travel-expense claim, independent of the monthly card cycle — same machinery, different date filter.

Built around the 18th, not the 1st

The card statement lands on the **18th**. That is the anchor for everything: the cycle closes, the statement is reconciled against what the app collected, discrepancies get five days to resolve, and the verified bundle goes to accounts on the **23rd–24th**.



CYCLE CONFIGURATION — CARD_ACCOUNTS

entity_name	"VIP Industries Limited"
label / last4	"HDFC Corporate" · 4821
statement_day	18 — the day the statement is issued
submit_day	23 — verified bundle goes to accounts
statement_password	to open HDFC's encrypted PDF (stored encrypted, never logged)
accounts_email	recipient(s) in the accounts department
min_receipt_amount	below this, receipts are auto-waived (default ₹500)

Cycles are materialised rows, not computed on the fly — a submitted cycle freezes, so changing `statement_day` next year never rewrites what accounts already received.

NUDGE LADDER

DAILY · 20:00 IST Charges from the last 48h with no receipt above the threshold. Silent if there is nothing to chase.

15TH · T-3 Full outstanding list, sorted by amount. "3 days to close · coverage 78%."

18TH · STATEMENT DAY Reconciliation report posted to chat: what tied out, what didn't, which refunds are still missing.

19TH–22ND Daily chase on the **remaining** gaps only, now measured against the statement rather than against the app's guess.

23RD Bundle built and posted for approval. Still nothing sends without your tap.

The statement is the truth — everything else is a claim

Until now the app's picture of a month came only from email alerts, which is a **lossy** source: alerts get missed, sync gaps happen, refunds arrive quietly. The statement is the bank's own authoritative ledger. Reconciling against it turns the submission from "what we think you spent" into something that **provably ties out**.

18TH · 01

Ingest
Gmail sync already matches hdfcbank senders, including emailstatements.cards@. The statement PDF is picked up automatically — no forwarding needed.

02

Decrypt
HDFC e-statements are password-protected. Opened with the stored password via a WASM qpdf build (works on serverless), then discarded from memory.

03

Parse
The decrypted PDF goes to Claude as a document, returning every line: date, description, amount, debit/credit, plus opening and closing balance.

04

Reconcile
Statement lines ↔ app transactions ↔ receipts. Every line lands in exactly one bucket below. Nothing is allowed to be unclassified.

THE SEVEN OUTCOMES, AND WHAT EACH TRIGGERS

BUCKET	EXPECTED	ACTION
Tied out	~85%	Statement line matches a transaction and carries a receipt. Nothing to do.
On statement, missing from app	a few	The gap you can't currently see. The alert email never arrived or a sync window was missed. The transaction is created from the statement line , then chased for a receipt. This alone justifies the feature.
In app, not on statement	occasional	Usually a pre-auth that never settled, or a charge that will land in the next cycle. Rolled forward rather than deleted, with a note.
Refund / credit landed	varies	Matched against the original charge and the expected-refund ledger. Marks that refund confirmed received .
Refund expected, not received	the point	An Amazon return or a disputed charge where the credit has not appeared . Escalated in chat by name and age — "₹4,199 Croma return, 41 days, still not credited". These are the ones that quietly get lost.
Chargeback / dispute	rare	Tracked as its own state through raised → provisional credit → resolved, so a provisional credit is never mistaken for a settled one.
Fees	every cycle	Annual fee, forex markup, GST on markup, late fee. Auto-categorised as fee, linked to the parent foreign charge where applicable, and never chased for a receipt.

THE CONTROL TOTAL — THE THING THAT MAKES THIS AUDITABLE

opening balance + debits – credits – payments = **closing balance**. The reconciliation must reproduce the statement's own closing figure **exactly**. If it doesn't, the difference is shown to the rupee and the cycle is blocked from submission until it's explained.

This is the single strongest claim the bundle can make to accounts: not "here are some receipts", but **"this reconciles to the bank's statement, and here is the proof line by line."** That is the difference between a submission that gets processed and one that generates a thread of questions.

WHAT CHAT LOOKS LIKE ON THE 18TH

Statement received · 18 Aug
47 lines · ₹1,58,204 · closing balance **ties out** ✓

✓ 41 matched & receipted

+ 2 charges the app never saw — added

↔ 3 refunds confirmed · ₹6,880

⚠ 1 refund still missing — Croma
₹4,199, 41 days

📄 3 fee lines auto-categorised

🔗 2 need receipts

📧 Chase Croma

5 days to the 23rd. Coverage **91%** against the statement.

Receipt Rail · VIP Industries Limited

12

A CSV, and every receipt file beside it

Accounts don't want a description of your receipts — they need the **files**, to attach against each payment line in their own system. So the deliverable is machine-readable first: one CSV row per statement line, and one receipt file per row, named so the two **pair by filename with no thinking required**.

THE PACKAGE

```
VIP_CorpCard_4821_2026-08.zip
├─ reconciliation_2026-08.csv ← the spine
├─ summary_2026-08.pdf      ← human-readable index
└─ receipts/
   ├─ R-001_2026-07-19_Indigo_12300.pdf
   ├─ R-002_2026-07-21_Taj_8940.jpg
   ├─ R-006_2026-08-12_IndianOil_3400.jpg
   ├─ R-014_2026-08-03_Lisboa-Rest_4751.jpg
   └─ ... 38 files
```

The filename is the join key. The receipt_file column in the CSV contains that exact string. Accounts sorts the CSV, opens the folder, and every row has its file — no cross-referencing a PDF, no asking you which receipt is which. **Originals are sent verbatim:** a vendor's e-invoice PDF stays a PDF, a photo stays the full-resolution photo.

CSV COLUMNS

stmt_line	Position on the bank statement — the audit anchor
txn_date · post_date	Purchase date and the date the bank posted it
merchant · descriptor	Cleaned name, plus the raw HDFC descriptor verbatim
amount_inr	The amount charged — authoritative, from the statement
currency · amount_orig · fx_rate	Populated for foreign charges; blank for domestic
type	purchase · refund · fee · payment
category	Existing category taxonomy
receipt_file	Exact filename in receipts/, or blank
receipt_status	attached · declared · waived · missing
declared_reason	Stated reason where no invoice exists
tax_id · tax_amount	GSTIN or EU VAT number, and recoverable tax
invoice_no · trip	Vendor invoice number; trip name for travel spend

UTF-8 with BOM so Excel opens Devanagari and accented European merchant names correctly on the first try — a small detail that otherwise generates an email.

WHY THE PDF STILL SHIPS ALONGSIDE

The CSV is for their system. Import, sort, filter, attach. It is the working artifact and the reason the whole package exists.

The PDF is for a human. Cover totals, the statement tie-out, the exception list with your declarations, and every receipt reproduced inline — for whoever reviews or files the claim without opening a spreadsheet.

Size discipline. Photos are capped at ~1 MB each (originals stay untouched in storage); vendor PDFs pass through as-is. If the ZIP still exceeds the ~40 MB mail limit it is split into part 1 / part 2, and a 30-day signed link to the full archive is always in the body.

Nothing leaves the building without your tap

THE 23RD, STEP BY STEP

1

Gate

Build is **blocked** unless the statement tie-out is zero. An unexplained difference stops the cycle rather than shipping a wrong number.

2

Build

CSV + summary PDF + receipts/ zipped; submissions row written as draft.

3

Preview

Bot sends the ZIP **to you** with headline numbers and the exception list.

4

Send

Resend → accounts, CC you. `resend_id` stored; bounces and delivery events logged.

📦 Ready · Cycle 18 Jul – 17 Aug

47 lines · ₹1,58,204 · **statement ties out** ✓

38 receipts attached · **93%** · 3 declared

VIP_CorpCard_4821_2026-08.zip · 22 MB

✓ Approve & send

✎ Fix first

👤 Recipients

Sent 23 Aug → accounts@vipindustries.com, cc you. Logged as submission #14.

EMAIL TO ACCOUNTS

To accounts@vipindustries.com · Cc you

Subject **Corp Card ••4821 — Aug statement, reconciled (47 lines, ₹1,58,204) + 38 receipts**

Hi team,

Attached is the reconciled statement for **18 Jul – 17 Aug 2026** on the corporate card ending 4821, verified against HDFC's statement dated 18 Aug.

Statement total	₹1,58,204.00
Reconciled to closing balance	✓ ties out
Lines · receipts attached	47 · 38 (93%)
Refunds confirmed received	3 · ₹6,880
Declared exceptions	3 · ₹700

The CSV's `receipt_file` column gives the exact filename in the **receipts/** folder — each row's receipt is attached in full, ready to file against the payment line.

📎 VIP_CorpCard_4821_2026-08.zip — reconciliation.csv · summary.pdf · receipts/ (38 files)

Sent via Resend from your existing verified domain. A bounce or delivery failure raises a Telegram alert — a failed send is never silent.

DASHBOARD · RECEIPTS TAB

93%

STATEMENT COVERAGE

Reconciliation table. Every statement line with its receipt thumbnail, match score and source.

3

AWAITING RECEIPT

Unmatched pool. Receipts with no home yet — drag onto a line to bind manually.

1

REFUND NOT RECEIVED

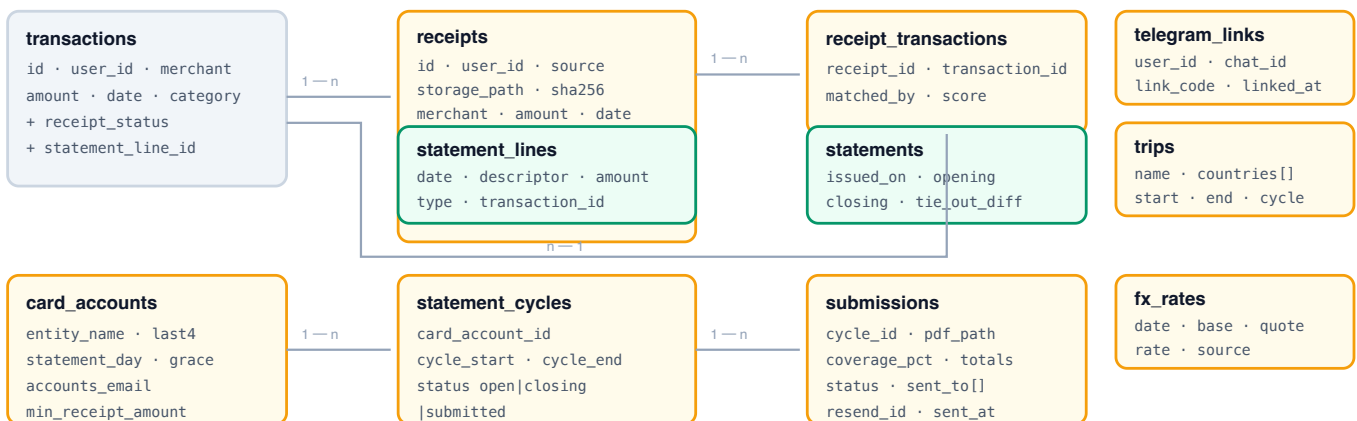
Refund ledger. Expected vs received, with age. The screen that recovers real money.

14

SUBMISSIONS SENT

Submission history. Past packages, recipients, and a re-send action.

Nine new tables, four new columns



MIGRATION — RUNS IN ONE SQL BLOCK

```

create table receipts (
  id          uuid primary key default gen_random_uuid(),
  user_id     text not null,
  source      text not null default 'telegram',
  tg_chat_id  bigint, tg_message_id bigint, tg_file_id text,
  storage_path text not null,
  mime        text, bytes int, sha256 text not null,
  doc_type    text,
  merchant    text, merchant_raw text,
  amount      numeric, tax_total numeric,
  receipt_date date, receipt_time text,
  -- international
  currency    text not null default 'INR',
  amount_inr  numeric, dcc_amount_inr numeric,
  fx_rate     numeric, fx_date date,
  country     text, city text, language text,
  tax_id      text, tax_id_type text, tax_breakdown jsonb,
  trip_id     uuid references trips(id),
  invoice_no  text, card_last4 text,
  extracted   jsonb, confidence numeric,
  status      text not null default 'pending'
  check (status in ('pending','matched',
    'unmatched','duplicate','rejected')),
  user_note   text,
  created_at  timestampptz default now(),
  unique (user_id, sha256)
);

-- the bank's own ledger; the spine of every submission
create table statement_lines (
  id          uuid primary key default gen_random_uuid(),
  statement_id uuid not null references statements(id),
  line_no     int, txn_date date, post_date date,
  descriptor  text, amount numeric not null,
  type        text check (type in ('purchase','refund',
    'fee','payment','chargeback')),
  transaction_id bigint references transactions(id),
  recon_status text -- tied|created|orphan|unexplained
);
create index on receipts (user_id, status, receipt_date desc);

alter table transactions
  add column receipt_status text default 'missing',
  add column declared_reason text;
  
```

DESIGN CHOICES WORTH DEFENDING

Join table, not a foreign key.

receipt_transactions allows one bill split across two charges and one charge covered by two documents (invoice + payment slip). Both happen.

Raw extraction is kept. extracted jsonb stores the model's full answer. When you improve the prompt in month three, you can re-derive fields from stored JSON without re-billing vision calls.

sha256 unique per user. The single cheapest defence against duplicate submissions — and it makes the bot's "you already sent me this" reply instant and free.

Materialised cycles. Submitted cycles freeze. Changing your statement day next year must not silently rewrite what accounts already received.

The statement outranks the app.

statement_lines is the spine of a submission — transactions becomes the enrichment layer hanging off it, not the source of truth. A line with no matching transaction creates one; a transaction with no line is rolled forward. statements.tie_out_diff must read zero before anything can be sent.

Currency is first-class, not an afterthought.

amount is always in the receipt's own currency; amount_inr is only an **estimate** — the authoritative rupee figure always lives on the transaction row. fx_rates is cached per currency-day so a lookup is never paid for twice.

Legacy has_receipt becomes a read-only view over receipt_status.

What gets built, concretely

API ROUTES

ROUTE	METHOD	AUTH	PURPOSE
/api/telegram/webhook	POST	secret header	Single entry point for all Telegram updates: photos, documents, commands, button callbacks. Idempotent on <code>update_id</code> .
/api/receipts	GET	session	Dashboard list with filters (status, cycle, unmatched only) + signed thumbnail URLs.
/api/receipts/[id]/match	POST	session	Manual bind / unbind to a transaction. Used by the dashboard drag-drop and the chat "wrong match" undo.
/api/receipts/[id]/file	GET	session	Issues a short-lived signed URL. Bytes are never served from the app itself.
/api/cycles	GET	session	Current + historical cycles with coverage stats.
/api/submissions	POST	session	Build a bundle for a cycle → returns draft submission + preview URL.
/api/submissions/[id]/send	POST	session	The only route that emails accounts. Requires an explicit approval token from the chat button or the dashboard.
/api/cron/tick	GET	CRON_SECRET	Daily dispatcher: <code>sync</code> → <code>rematch</code> → <code>nudge</code> → <code>cycle-close</code> → <code>monthly-report</code> , each guarded by its own date condition. Supports <code>?job=</code> for manual runs.

TOOL & SERVICE INVENTORY

TOOL	STATUS	ROLE	NOTES / SETUP
Telegram Bot API	new	Capture + nudge channel	Create via @BotFather, webhook with <code>secret_token</code> . Free. 20 MB file cap.
Sonnet 5 + GPT-5.6 Terra Opus 5 tiebreak	both keys held	Dual-read extraction	Consensus routing for accuracy (p.07). <code>lib/sync.js</code> stays on Claude, untouched.
Supabase Storage	new bucket	Receipt object store	Private bucket <code>receipts</code> . Needs <code>SUPABASE_SERVICE_ROLE_KEY</code> for server-side writes.
Supabase Postgres	have	All new tables	One migration, no changes to existing rows beyond added columns.
pdfkit + pdf-lib	1 new dep	Bundle assembly	<code>pdfkit</code> already installed; <code>pdf-lib</code> added to merge vendor e-invoice PDFs.
sharp	1 new dep	Image prep	EXIF auto-rotate, HEIC→JPEG, downscale before vision and before bundling. Big win on both cost and file size.
Frankfurter (ECB)	new	Historical FX rates	Free, no API key, ECB daily reference rates. Cached in <code>fx_rates</code> — a few dozen calls a year.
Resend	have	Delivery to accounts	Already wired in <code>lib/monthly-report.js</code> . Free tier covers this comfortably.
Vercel Cron + Functions	have	Scheduling + compute	Consolidate to one dispatcher cron (p.03; alternatives on p.16). <code>maxDuration = 300</code> already proven on the <code>sync</code> route.

MONTHLY RUNNING COST (~120 RECEIPTS, 1 CYCLE)

Extraction · dual read + 15% tiebreak	≈ \$2.30
FX rates · Frankfurter	\$0.00
Object storage (≈40 MB/mo)	\$0.00 (free tier)
Telegram Bot API	\$0.00
Resend (≈ 5 emails/mo)	\$0.00 (free tier)
Compute + scheduling	\$0.00 (existing)
Total added	≈ \$2.30 / month

Receipt Rail · VIP Industries Limited

NEW ENVIRONMENT VARIABLES

```
TELEGRAM_BOT_TOKEN
TELEGRAM_WEBHOOK_SECRET
TELEGRAM_ALLOWED_CHAT_IDS # comma list
OPENAI_API_KEY # second opinion
SUPABASE_SERVICE_ROLE_KEY # server only
ACCOUNTS_EMAIL # default recipient
RESEND_FROM # verified sender
```

Two constraints, and what to replace them with

A · SCHEDULING — REPLACING THE 2-SLOT VERCEL CRON LIMIT

OPTION	COST	GRANULARITY	TRADE-OFF
One dispatcher cron SIMPLEST	₹0	daily	Stay on Hobby. One <code>/api/cron/tick</code> branching on the date. Zero new vendors — but one run a day, so every nudge fires at the same fixed hour.
Supabase pg_cron + pg_net RECOMMENDED	₹0	per minute	Schedules live in the database you already run, calling your routes over HTTP. Unlimited jobs, minute granularity, unaffected by any Vercel plan change — and the schedule is SQL, so it lives in the migration.
Upstash QStash BEST FIT	free ≤500 msg/day	per minute	Scheduler and job queue in one. The Telegram webhook enqueues the vision job and returns 200 instantly, with retries, backoff and dedupe you didn't write — removing the "ack fast, work later" hack entirely.
Cloudflare Workers Cron	₹0	per minute	Free and dependable, but adds a second deployment target to keep in sync.
GitHub Actions schedule	₹0	~5–15 min drift	Zero infra, but runs are best-effort and can be delayed or dropped. Fine for a nudge, wrong for a cycle close.
Inngest / Trigger.dev	free tier	per minute	Durable multi-step workflows with retries and replay — good for ingest → extract → match, but more machinery than day one needs.
Vercel Pro	\$20/mo	per minute	Removes the limit and changes nothing else. Only worth it if you want Pro anyway.

Recommendation: **pg_cron** for schedules — no new vendor, minute granularity, versioned in SQL — plus **QStash** for the async ingest queue if you'd rather not hand-roll retries. Both free at this volume. Keep the dispatcher route either way: a single `/api/cron/tick?job=` entry point makes the scheduler swappable in one line and keeps every job manually runnable.

B · SUPABASE — THE AUTH-ISSUER PROBLEM BEHIND THE ANON KEY

Why RLS is effectively off today.

The app authenticates with **NextAuth + Google**, not Supabase Auth. Supabase's `auth.uid()` reads a JWT signed by **Supabase's own issuer** — a NextAuth session carries no such token, so any policy written against it denies everything. The config that worked became "anon key + permissive policies", which is exactly why the daily cron can read every user's rows. Not a bug anyone introduced — the default outcome of mismatched issuers.

Fix: service-role + app-layer scoping. **DO THIS**

All DB access stays server-side on the service-role key; every query keeps its `.eq('user_id', ...)` filter exactly as today; RLS is set to **deny-all for anon** as defence in depth. About an hour of work, zero behaviour change, and the cron's cross-user read becomes explicit and intentional rather than accidental.

If you want real RLS later. Sign a Supabase-compatible JWT in the NextAuth callback carrying `sub = user_id`, using the Supabase JWT secret — then `auth.uid()` works. ~30 lines, only needed if the browser should query Supabase directly.

Leaving Supabase entirely? Neon (Postgres, generous free tier, on the Vercel Marketplace) is the closest swap — but it migrates a working database for no functional gain. Fix the key model instead.

C · STORAGE — WHERE RECEIPT IMAGES ACTUALLY BELONG

OPTION	COST	NOTES
Supabase Storage	1 GB free · egress billed	Simplest — same client, same auth. But 7-year retention at ~36 MB/month crosses the free tier in about 2.5 years .
Cloudflare R2 BEST LONG-TERM	10 GB free · \$0.015/GB · zero egress	No egress charge at all, which matters because every bundle build re-reads 40 images. S3-compatible, presigned URLs.
Vercel Blob	private blobs · egress billed	Nothing new to configure: same platform, same env, signed URLs built in. Cost lands between the other two.
Verdict	Ship on whichever is fastest, behind <code>lib/storage.js</code> — moving to R2 later costs one file, not a migration.	

This layer handles company financial records

Receipts carry GSTINs, card tails, vendor details and, for travel, personal itineraries. The bar is higher than for the dashboard, because this data leaves the system and lands in an accounts inbox.

CONTROLS

Webhook auth	Telegram secret_token header compared in constant time; unknown chat_id silently dropped and logged. No open POST endpoint.
Account binding	One-time codes from Settings, single-use, 15-minute expiry, one chat per user. Unlink button revokes instantly.
Object storage	Private bucket. No public URLs anywhere. Access exclusively via signed URLs with a 15-minute TTL, issued only to the owning session.
Key hygiene	Service-role key is server-only and never reaches the browser bundle. Bot token never logged, never echoed in errors.
RLS	Row-level security on every new table keyed to user_id; cron paths use the admin client explicitly and are the only cross-user readers.
Send gate	Outbound email requires an explicit approval action tied to a submission id. No code path emails accounts automatically.
Audit trail	app_logs records every extraction, match, unmatch, declaration, bundle build and send — who, when, what score.
Retention	Configurable. Default: originals kept 7 years (statutory GST record retention in India is 6 years from filing), extraction JSON kept indefinitely, Telegram file ids purged after binding.

THREATS CONSIDERED

- Someone finds the bot.** They can message it, but with no valid link code and an unknown chat id, every update is dropped before any work is done. No storage write, no vision call, no cost.
- Webhook replay.** update_id dedupe makes handling idempotent; a replayed photo resolves to the same sha256 and returns the existing match.
- Wrong bundle to the wrong recipient.** Recipients come from card_accounts, are shown in the approval card before sending, and changing them is a separate deliberate action.
- Personal receipt sent by mistake.** The bot flags any receipt whose extracted card_last4 disagrees with the configured company card and asks before filing it. Mixed personal spend on a company submission is a real audit problem.
- Vision provider exposure.** Receipt images are sent to Anthropic for extraction — the same trust boundary your bank alert emails already cross today in lib/sync.js. Worth stating explicitly to whoever signs off internally.
- Pre-existing issue this surfaces.** The current app authenticates Supabase with the anon key and the daily cron reads every user's rows — which only works because RLS isn't enforcing isolation. Phase 0 fixes that properly rather than building more on top of it.

Which chat channel — and why Telegram wins

You asked for "Telegram or iMessage". They are not equivalent engineering problems. Here is the honest comparison.

CRITERION	TELEGRAM RECOMMENDED	IMESSAGE	WHATSAPP BUSINESS CLOUD
Official bot API	Yes — first-class, free, documented	No. Apple offers no bot/automation API for iMessage.	Yes, Meta Cloud API
Infrastructure	Webhook → existing Vercel function. Nothing else to run.	Requires an always-on Mac running BlueBubbles/AppleScript with Full Disk Access, exposed to the internet.	Webhook → Vercel, plus Meta Business verification
Bot-initiated messages	Unlimited once you've started the chat — nudges just work	Possible via the bridge, fragile	Blocked outside a 24h window unless you use paid approved templates
Buttons / inline UI	Inline keyboards, callbacks, editable messages	Plain text only	Limited quick-replies
File handling	Photos + documents, 20 MB, direct download URL	Works, via the bridge's local filesystem	Media API, 16 MB
Ongoing cost	₹0	Cost of keeping a Mac powered on 24/7	Per-conversation charges on business-initiated messages
Reliability risk	Low	High — unofficial, breaks on macOS updates, Apple can shut it down	Medium — Meta policy changes, template approvals
Build effort	~1.5 days for the full channel layer	~4–5 days plus permanent Mac ops burden	~3 days plus business verification lead time

Recommendation: build Telegram first

It is the only option where the channel is a solved problem and all the effort goes into the parts that actually create value — extraction, matching, bundling. You install one free app and the entire nudge/button experience on page 04 becomes possible.

The channel is deliberately abstracted. `lib/telegram.js` sits behind a thin `ChannelAdapter` interface (`receiveUpdate`, `sendMessage`, `sendDocument`, `editMessage`). Adding iMessage or WhatsApp later means writing one adapter, not touching the pipeline.

IF YOU TRULY WANT IMESSAGE

Two routes exist, both with the same fundamental caveat — they impersonate a logged-in Mac rather than using a supported API:

- **BlueBubbles server** on a Mac mini — REST API over the local Messages database, webhook out to your Vercel app.
- **chat-adapter-imessage** (community adapter for the Chat SDK) — same underlying approach, nicer abstraction.

Either way you own a machine that must never sleep, must keep Full Disk Access after every macOS update, and can break without notice. Sensible as a **Phase 7 convenience**, not as the foundation of a compliance workflow.

Middle path if the Telegram app is a non-starter: keep Telegram as the engine and add an **email-in** adapter — forward or AirDrop-to-Mail any receipt to a dedicated address, the same pipeline ingests it. You lose buttons and nudges, but capture still works from any device with zero new apps.

Failure modes, named in advance

RISK	LIKELIHOOD	IMPACT	MITIGATION BUILT INTO THE DESIGN
Wrong auto-match slips into a submission	Medium	High	Conservative thresholds (score ≥ 75 and 15-point gap), one-tap undo on every match, score + reason stored, bundle preview shown before sending.
Vision misreads the total (tip line, rounding)	High	Medium	Prompt insists on amount-charged-to-card; arithmetic cross-check; and the bank amount always wins — the receipt is evidence, the transaction is truth.
Receipt photographed before the bank alert arrives	High	Low	Deferred pool + rematch after every sync. Expected common case, handled by design, invisible to you.
You stop sending photos after week 3	High	High	The single biggest risk is behavioural, not technical. Daily nudges with a 2-second reply path, and coverage % as a visible score, exist specifically for this.
Gmail refresh token expires (already happened once)	Medium	Medium	Existing <code>auth_failed</code> logging is extended to alert you in Telegram instead of only writing to <code>app_logs</code> .
Bundle too large / bounces	Medium	Low	Downscale to <10 MB target, plus a signed link to full-resolution originals in the mail body.
Cron slot limit on the current Vercel plan	Certain	Low	Single dispatcher tick with date-branching. Solved in Phase 0.
Accounts wants a different format	High	Low	Show them a sample bundle in Phase 5 before finalising layout. Cover/index/exception structure is data-driven and cheap to restyle.
Telegram webhook downtime	Low	Low	Telegram retries failed deliveries; <code>update_id</code> dedupe makes retries safe. Nothing is lost, only delayed.

THE ONE I'D WATCH HARDEST

Adoption, not accuracy. A reconciliation tool with 99% match accuracy and 40% capture rate is worse than one with 90% accuracy and 95% capture. Every design choice above — 2-second send, no app-switching, buttons instead of typing, daily chase, one-tap declarations — is aimed at capture rate. Treat **coverage %** as the product's north-star metric, not match precision.

DELIBERATELY OUT OF SCOPE FOR V1

- Multi-user / whole-team rollout — built user-scoped, but shipped for one cardholder first
- Approval workflow inside accounts (they keep their own process)
- ERP/Tally push — the CSV is the integration seam
- Automatic GST input-credit claiming — the summary page informs, it doesn't file
- Mileage / per-diem / cash expenses — card charges only

Thirteen answers unblock the entire build

Everything else in this document is decided. These are the ones only you can answer — most take a few seconds.

#	QUESTION	DEFAULT IF YOU DON'T ANSWER	BLOCKS
1	Password for the HDFC e-statement PDF (stored encrypted, never logged)	Statement parsing can't run without it	P5
2	Accounts department email address (and anyone to CC)?	Placeholder; nothing sends until set	P5
3	Last 4 digits of the corporate card	Skip the card-tail signal (–10 pts of match confidence)	P3
4	Below what amount is a receipt not required?	₹500 auto-waived	P4
5	Telegram — will you install it? If not, say so now and I build the email-in adapter instead.	Telegram	P1
6	Auto-send the bundle at cycle close, or always ask you first?	Always ask	P5
7	Does accounts want a specific format? Worth one email to them before P5.	The layout on page 11	P5
8	Any personal spend on this card that must be excluded from submissions?	Everything is submitted	P4
9	Should this stay single-user, or will other VIP cardholders use it later?	Built user-scoped, shipped for you alone	P0 sizing
10	Is the statement emailed to this same Gmail account? If it goes elsewhere, forwarding takes one rule.	Assumed same inbox	P5
11	Which countries and rough dates was the Europe trip?	Auto-detected from the receipts	P3
12	HDFC forex markup on this card — standard 3.5%, or a negotiated corporate rate?	3.5% + 18% GST on the markup	P3
13	Does VIP reclaim EU VAT on business travel?	Summary included, ignored if irrelevant	P5

IF YOU SAY GO, THE FIRST COMMIT IS PHASE 0

- 1 Add SUPABASE_SERVICE_ROLE_KEY, create the private receipts bucket, enable RLS on new tables.
- 2 Run the single migration — 9 tables, 4 columns on transactions, indexes.
- 3 Collapse vercel.json to one /api/cron/tick dispatcher and verify sync + monthly report still fire.
- 4 Create the bot with @BotFather, register the webhook with a secret token, link your chat.

End of day 3 you can message the bot and get your live cycle status back. Everything after that is incremental.

HOW YOU'LL KNOW IT'S WORKING

- | | |
|----------------|---|
| Week 1 | Coverage on the open cycle climbs past 50% with no month-end effort. |
| Week 4 | First bundle reaches accounts and comes back with zero follow-up questions. |
| Month 3 | You've stopped thinking about receipts — the only reminder is a nudge you clear in ten seconds. |

The measurable claim: from ~45 minutes of month-end reconstruction per cycle — plus whatever receipts were simply lost — to ~3 minutes of taps spread across the month, with a coverage number you can defend to finance.